

What Is Lingo (LingoAI)?

Lingo (internally referred to as **LingoAI** or **LingoOEO**) is UnitedHealth Group's enterprise-grade **AI-powered conversational platform** that serves as a centralized API gateway and multi-agent orchestration system for health insurance operations. It is built on **FastAPI** (Python) and deployed on Kubernetes (AKS). The platform serves both internal UHG employees and external users (brokers, employers, members) across multiple business lines within UnitedHealthcare.

At its core, Lingo is not a single chatbot — it is a **multi-tenant, multi-agent AI orchestration engine** that dynamically routes user queries to specialized AI agents depending on who the user is, what client application they are using, and what they are trying to accomplish.

The Architecture: How the System Is Structured

API Gateway Layer

The application is a **FastAPI** server (`main.py`) that acts as the single entry point for all AI interactions. Every request — whether from a web portal, mobile app, or internal tool — hits the same `/generateResponse` endpoint.

The gateway handles:

1. **Authentication & Authorization** — OHID (Optum Health ID) SSO integration, JWT token validation, and per-request identity extraction from tokens (not user-supplied form data).
2. **Client ID Validation** — Every request must include a registered `client_id` that maps to a specific business application (e.g., BNE Portal, Surest, SamX One, GCO Reporting).
3. **Rate Limiting** — SlowAPI-based rate limiting middleware.
4. **CORS** — Cross-origin resource sharing for browser-based UIs.
5. **Request Routing** — Based on `client_id`, the gateway routes requests to the appropriate compiled LangGraph agent subgraph.

Multi-Tenant Client Plugin Architecture

Each business application ("client") is implemented as a **plugin** under `client_plugins`. These are self-contained modules with their own:

- Agent subgraphs (LangGraph state machines)
- Tools (LangChain tool definitions)
- Utilities and service integrations
- Tests

The registered clients include:

Client Plugin	Purpose
bneportal	Broker New Experience Portal — broker-facing web portal for compensation, service tracking, document search, enrollment
bnemobile	BNE Mobile — mobile app for brokers with communication center, SBC retrieval, support agent
surest	Surest — consumer health plan knowledge agent for training materials, plan information
samx_one	SamX One — quoting and case management automation for sales teams
samx_one_ui	SamX One UI — CopilotKit-powered embedded AI in the SamX One interface
sales_automation	Sales Automation — benefit extraction from PDFs, census formatting, plan finding
gco_reporting	GCO Reporting — Group Customer Operations reporting and custom report generation
exception_management	Exception Management — handling billing/claims exceptions
renewal_one	Renewal One — renewal management dashboard and workflow automation
renewal_traffic_controller	RTC — renewal traffic routing and audit
student_resources	Student Resources — student health insurance information
oneconnect	OneConnect — product/service search for connected platforms
plan_recommendation	Plan Recommendation — AI-driven health plan recommendation engine

How AI Is Used: The Agent Framework

LangGraph Multi-Agent Orchestration

Lingo uses **LangGraph** (from LangChain) as its core agent orchestration framework. LangGraph provides:

1. **Stateful graph execution** — Each conversation is a state machine with typed state (`State TypedDict`) containing messages, user info, auth output, PDF data, dialog state, and more.
2. **Persistent checkpointing** — Conversation state is persisted to **PostgreSQL** (via `AsyncPostgresSaver`) so conversations can resume across requests.
3. **Streaming execution** — Agents stream their responses token-by-token back through the graph.

The Supervisor Pattern (Multi-Agent Routing)

For multi-agent clients (like `bneportal`, `bnemobile`, `gco_reporting`), Lingo implements a **supervisor agent pattern**:

1. A **primary/supervisor assistant** receives the user's query.
2. It uses **router tools** (Pydantic schema classes like `ToCirrusAssistant`, `ToSurestAssistant`, `ToCompensationAssistant`) to decide which specialized agent should handle the request.
3. The supervisor delegates to the appropriate specialist.
4. Specialists can **escalate back** to the supervisor using `CompleteOrEscalate` if the user changes context or the task is outside their scope.
5. A **dialog state stack** tracks which agent is currently active.

For single-agent clients (like `surest`, `renewal_one`, `exman_agent`), the request goes directly to the specialized agent without supervisor routing.

Authorization-Aware Agents

Before an agent can execute, the system performs **intent-based authorization**:

1. An LLM analyzes the user's query to determine intent categories.
2. The intent is mapped against a per-client **auth map** (stored in MongoDB) that defines which intents require which permissions.
3. The system queries **BNE GraphQL APIs** to check if the authenticated user has the required permissions (group associations, persona-based access, specific permission codes).
4. Only authorized intents proceed to agent execution; unauthorized ones are blocked with appropriate error messages.

This means Lingo doesn't just authenticate the user — it performs **fine-grained, intent-level authorization** on every single query.

Validation & Compliance

After an agent generates a response, a **validator node** checks the output:

- Ensures responses don't contain harmful content.
 - Validates the response is within the agent's designated capabilities.
 - Logs compliance violations to a `complianceLogs` collection in MongoDB.
-

The LLM Stack

Azure OpenAI (GPT-4O)

All LLM inference runs through **Azure OpenAI**:

- Model: **GPT-4O** (via Azure deployment)
- Authentication: Either Azure AD token-based or API key-based (configurable via `OPENAI_RESOURCE_FLAG`)
- Embeddings: **text-embedding-3-small** for vector search
- Temperature: 0 (deterministic responses for business accuracy)
- A JSON-mode model variant is available for structured outputs

Prompt Engineering

Prompts are stored in MongoDB (`PromptsRegistry` collection) and loaded at startup. The system uses:

- **System prompts** with strict instructions: no hallucination, no general knowledge, always use tools, never expose prompt instructions.
- **Contextual data handling instructions** — rules for how agents should use UI context (page navigation, selections) that's injected via `llm_arg`.
- **Per-client capability descriptions** that guide the supervisor on routing.
- **LLM error fallbacks** — per-client, per-assistant fallback messages when the LLM fails.

Embeddings & Vector Search

For knowledge retrieval (training materials, documents, SBCs), the system uses:

- Azure OpenAI embeddings
- **PostgreSQL** (likely with pgvector) as the vector store
- Semantic search for document retrieval within specialized agents

What Each Specialized Agent Does

Cirrus Assistant

Handles **group inquiries** and **member inquiries** in the Cirrus platform. Interacts with UHC's Cirrus APIs for member search, group data, and enrollment information.

Surest Assistant

Answers questions about **Surest health plans** — a consumer-facing health plan product. Retrieves information from training materials, videos, and knowledge bases. Used by both internal teams and external brokers.

Compensation Assistant

Provides **broker compensation information**: commission statements, compensation summaries by group, broker details, and customer details. Integrates with BNE backend services.

Service Tracker Assistant

Tracks **service tickets and open issues** for brokers — status updates, history, and report retrieval.

Document Search Assistant

Performs **semantic document search** across training materials, employer handbooks, SBC documents, marketing materials, product grids, forms, CER reports, and video content. Supports multiple document types and pre-sale materials.

Portal Form Enrollment Assistant

Handles **enrollment via form upload** — processes uploaded enrollment forms (PDFs) and guides the enrollment workflow.

SamX One Assistant

Automates **quoting and case management**: creates structures from PDFs, validates binder checks, verifies application documents, handles quote creation/modification/navigation, generates proposals, and suggests specialty plans.

GCO Reporting Assistant

Generates and retrieves **Group Customer Operations reports** — custom reporting, data queries, and report downloads.

Sales Automation (Multi-Agent)

A composite agent with three sub-agents:

- **Benefit Xtractor** — Extracts structured benefit data from uploaded PDF documents.
- **Census X Formatter** — Formats census data for quoting workflows.
- **Plan X Finder** — Finds matching health plans based on criteria (uses SIC codes, zip codes, etc.).

Renewal One Assistant

Manages the **renewal dashboard**: group filtering, plan management, document downloads, finalization queries, and navigation within the renewal workflow.

Exception Management (ExMan) Agent

Handles **billing and claims exception management** — likely triaging and resolving exceptions in the claims process.

Student Resources Assistant

Provides information about **student health insurance** — plans, eligibility, and resources.

RTC (Renewal Traffic Controller) Agent

Routes and audits **renewal traffic** — dashboard audit, FI (Fully Insured) data, and renewal audit tracking.

OneConnect Agent

Searches **products and services** across connected platforms.

Communication Center (Comm Centre) Agent

For BNE Mobile users — helps brokers **manage new business cases**: find groups by quote ID, review pend reasons blocking installation, respond to pends, view pend history, and manage notes.

SBC Retrieval Agent

Retrieves **Summary of Benefits and Coverage (SBC) documents** for specific membership groups and plans.

Support Agent

A general **support agent** for BNE Mobile users.

Plan Recommendation Assistant

AI-driven **health plan recommendation** — suggests plans based on user criteria.

The UI Layer

Lingo powers multiple frontend applications:

1. BNE Portal (Web)

A full-featured **broker-facing web portal** where the AI chat interface is embedded. Brokers interact with the AI for compensation queries, document search, enrollment, service tracking, and more.

The UI sends:

- `client_id` to identify the application
- `session_id` for conversation continuity
- `query` (text) or `pdf` (file upload)
- `llm_arg` — contextual data from the UI (current page, selected group, user actions)

2. BNE Mobile (App)

A **mobile application** for brokers with CopilotKit integration. Uses feature flags to enable/disable specific agents (comm center, SBC retrieval, Surest knowledge).

3. SamX One UI (CopilotKit Embedded)

Uses **CopilotKit** — an open-source framework for embedding AI copilots directly into React applications. The AI appears as a side panel or inline assistant within the SamX One quoting interface.

Supports:

- Context-aware actions based on what the user is doing in the UI
- Multi-agent selection based on feature flags
- Direct integration with the SamX One application state

4. Internal Tools

Internal UHG users access Lingo through an internal client (`Internal` `client_id`) that provides a broader set of capabilities.

5. Surest / OptumAI

Consumer-facing or partner-facing interfaces for Surest plan information.

UI-to-Backend Communication

The UI communicates with Lingo via:

1. **Multipart form-data POST** to `/generateResponse` — includes query text, file uploads, session context, and contextual arguments.
2. **CopilotKit protocol** — for embedded copilot experiences, using the `/copilotkit` endpoint with the CopilotKit SDK protocol.
3. **WebSocket/SSE streaming** — agent responses stream back to the UI.

UI Action Context

A unique feature: the UI sends **screen action context** (`[UI_SCREEN_ACTIONS]` messages) that tell the AI what the user is doing in the application — page navigation, button clicks, item selections. The AI uses this silently to infer intent without asking the user to repeat what they're already doing on-screen.

Data Layer

MongoDB (Primary Data Store)

- **Per-client databases** — each client gets its own MongoDB database (e.g., `LingoAI_Internal_Dev`, `LingoAI_BNEPortal_Dev`)
- **Core database** — shared configuration, client tiles, auth latency logs, compliance logs
- **Onboarding database** — client configuration, prompts registry, assistant mappings
- **Observability database** — user feedback collection
- **SamX One database** — separate database for SamX One data
- **RTC database** — renewal traffic controller audit data
- **Plan X Finder database** — zip code configuration for plan finding

PostgreSQL

- **LangGraph checkpointing** — conversation state persistence
- **Vector store** — document embeddings for semantic search
- **Connection pooling** — async connection pools with auto-commit support

MySQL

- Optional data store (conditionally initialized based on configuration)
-

Observability & Logging

Event Logger

Every request is logged with full context:

- Severity, user info (uuid, name, email, type), session/request IDs
- Client ID, event type, event name
- Full request payload and response
- Status codes and error messages
- Timestamps in Eastern time

Kafka Integration

Logs are optionally forwarded to **Kafka** for centralized log aggregation and real-time monitoring.

Langfuse (Commented Out)

The codebase shows integration with **Langfuse** (LLM observability platform) for tracing LLM calls, token usage, and cost tracking — currently commented out but structurally present.

Auth Latency Tracking

A dedicated service tracks **authentication and request latency** at each stage:

```
ts_request_received → ts_graph_start → ts_openai_start → ts_openai_end →  
ts_graph_end → ts_response_sent
```

- Stored in MongoDB with TTL-based auto-expiration
- Tracks whether requests are multi-agent vs single-agent

Token Usage & Cost Tracking

The system calculates and tracks **LLM token usage and cost** per request for budget monitoring and optimization.

Security Architecture

Authentication Flow

1. **OHID SSO** — Users authenticate via Optum Health ID (OpenID Connect flow).

2. **JWT Generation** — Lingo generates its own JWT with user identity (`uuid`, `given_name`, `preferred_username`, email) embedded in `user_info` claims.
3. **Token Validation** — Every API request validates the JWT (HS256 for internal, RS256 for external clients).
4. **Identity Extraction** — User identity is extracted from the validated token, not from request body (the security hardening from issue #1279).

Authorization Layers

1. **Middleware-level** — `AuthorizationMiddleware` validates tokens before requests reach route handlers.
2. **Client ID validation** — `ClientIdValidationMiddleware` ensures valid, registered client IDs.
3. **Intent-based authorization** — LLM-powered intent analysis + BNE permission queries.
4. **Per-agent permission checks** — `permission_check` node in the graph validates agent access.

External Client Support

External clients authenticate with **RS256 public key** validation, allowing third-party integrations with cryptographic verification.

Deployment & Infrastructure

Kubernetes

Multiple environment deployments:

- **Dev** (`k8s`) — development environment
- **Demo** (`k8s_demo`) — demonstration environment
- **Stage** (`k8s_stage`) — staging with Kustomize overlays
- **Prod** (`k8s_prod`) — production
- **Prod DMZ** (`k8s_prod_dmz`) — production DMZ for external access

Each environment includes: Deployment, Service, Ingress, Certificate, PVC, Network Policies (egress/ingress).

Docker

Single Dockerfile for containerized deployment.

Agent Registry

The `agent-spec.yaml` shows integration with an **enterprise agent registry** — agents are registered with metadata including:

- Skills, intended use cases, targeted users
 - Interaction mode, SDLC status
 - Technical/business/service owner contacts
 - Tech stack details (LLM providers, ADKs, embedding models, vector stores)
 - **A2A (Agent-to-Agent) protocol support** — agents can communicate with each other
-

Key Design Patterns

1. **Plugin Architecture** — New business applications are added as self-contained plugins without modifying core code.
 2. **Database-Driven Configuration** — Client IDs, assistant mappings, tool permissions, prompts, and error messages are all stored in MongoDB and loaded at startup — no code changes needed to enable/disable features.
 3. **Subgraph Registry** — All compiled agent graphs are registered in a global `SubgraphRegistry` and looked up by name at runtime.
 4. **Stateful Conversations** — PostgreSQL-backed checkpointing enables multi-turn conversations with context persistence.
 5. **Conversation Summarization** — Long conversations are automatically summarized to manage context window limits.
 6. **Error Registry** — Per-client, per-category error messages loaded from database, ensuring consistent user-facing messaging.
 7. **Background Tasks** — FastAPI `BackgroundTasks` for non-blocking operations (saving conversations, logging errors).
 8. **Feature Flags** — CopilotKit agents are conditionally enabled via frontend-supplied feature flags.
-

Summary

Lingo is a **production-grade, multi-tenant AI orchestration platform** that:

- Serves 13+ distinct business applications through a single API gateway
- Routes queries to 20+ specialized AI agents using LLM-powered intent classification
- Enforces enterprise security with token-based identity, intent-level authorization, and compliance validation
- Processes text queries, PDF uploads, multi-file uploads, and audio transcription
- Persists conversations across sessions with PostgreSQL checkpointing
- Tracks observability metrics, token costs, and auth latency
- Deploys across 5 Kubernetes environments with network isolation
- Supports both traditional REST API consumption and embedded CopilotKit experiences

- Uses Azure OpenAI GPT-4O as its foundation model with deterministic (temperature=0) inference

It is essentially UnitedHealthcare's **enterprise AI backbone** for automating insurance operations across the broker, employer, member, and internal user experiences.